

**ВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего профессионального образования
«Вятский государственный университет»
(ФГБОУ ВПО «ВятГУ»)
Факультет автоматики и вычислительной техники
Кафедра электронных вычислительных машин**

**Методические указания
к самостоятельным и лабораторным работам
по дисциплине «Базы данных»**

**Лабораторная работа №2_1.
Основы DML-запросов в PostgreSQL**

Цели лабораторной работы:

- освоить основные варианты DML-запросов в PostgreSQL;
- научиться создавать SQL-скрипты для заполнения таблиц данными;
- научиться использованию команды `update` и `delete`,
- научиться работать с представлениями.

Задание на лабораторную работу:

При выполнении работы используется БД, созданная в лабораторной работе №1.

Нужно выполнить следующие задания:

1. Создать и выполнить SQL-скрипт, который будет заполнять таблицы данными. Нужно добавить не менее 3-5 строк в каждую таблицу. (**команда `insert`**).
2. Создать и выполнить SQL-скрипт изменения существующих строк таблицы (**команда `update`**).
3. Создать и выполнить SQL-скрипт удаления строк из таблицы используется (**команда `delete`**).
4. Создать и выполнить SQL-скрипт используется для получения строк из таблиц (**команда `select`**). Выполнить все примеры `select` с разными предложениями и соединениями для своей предметной области(БД).
5. Создать представления с выборкой из нескольких таблиц, в которых собираются данные из самой таблицы и других, на которые она ссылается. Выборка из любого представления должна давать полную и осмысленную информацию по сущностям. Хотя бы одно из представлений должно быть сделано с использованием соединений (`join`) в запросе.

Отчет по лабораторной работе должен содержать:

- созданные скрипты с комментариями (скрипт для заполнения таблиц данными и скрипты, создающие представления);
- примеры результатов выборок из созданных представлений.

Ниже показан скрипт, который был создан в процессе выполнения лабораторной работы 1. Для выполнения данной лабораторной работы используем данную структуру БД.

```
--Создание таблицы ГРУППЫ
create table public.group(
    id bigserial primary key,
    name varchar(30) not null unique,
    number_of_students int not null check (number_of_students >= 1)
);

--Создание таблицы СТУДЕНТЫ
create table public.student(
    id bigserial primary key,
    name varchar(30) not null,
    surname varchar(30) not null,
    middle_name varchar(30),
    id_group bigint not null references public.group(id),
    address varchar(100),
    course int check (course >= 1 and course <=5)
);

--Создание таблицы ПРЕДМЕТЫ
create table public.subject(
    id bigserial primary key,
    name varchar(50) not null unique,
    number_of_hours int not null check (number_of_hours > 0)
);

--Создание таблицы УСПЕВАЕМОСТЬ
create table public.progress(
    id_student bigint not null references public.student(id),
    id_subject bigint not null references public.subject(id),
    rating int check (rating >= 2 and rating <=5)
);

--Создание типа перечисления для обозначения должности преподавателя
create type public.position_type_enum as enum (
    'docent', 'professor', 'senior lecturer', 'instructor');

--Создание таблицы ПРЕПОДОВАТЕЛИ
create table public.teacher(
    id bigserial primary key,
    name varchar(30) not null,
    surname varchar(30) not null,
    middle_name varchar(30),
    position public.position_type_enum not null default 'instructor',
    phone_number varchar(30)
);

--Создание типа перечисления для обозначения типа проводимого занятия
create type public.occupation_type_enum as enum (
    'lecture', 'laboratory work', 'practical lesson', 'seminar');

--Создание таблицы ТИП ЗАНЯТИЯ
create table public.type_of_occupation(
    id_subject bigint not null references
public.subject(id),
    id_teacher bigint not null references
public.teacher(id),
    occupation_type public.occupation_type_enum not null
```

```
        default 'lecture'  
    );
```

1. Команда insert

Создать и выполнить SQL-скрипт, который будет заполнять таблицы данными. Нужно добавить не менее 3-5 строк в каждую таблицу.

Команда insert используется для вставки строк в таблицу. За один запрос можно вставить ноль и более строк. Она может как добавлять строки с указанными явно значениями, так и добавлять строки, которые были получены из дополнительного запроса (см. [пункт «Подзапросы и предложение with»](#)).

Здесь будут рассмотрены базовые способы применения команды insert. *Подробнее о синтаксисе команды insert можно прочитать в документации.*

-- Пример 1

```
-- Вставляет новую строку в customer, id проставляется автоматически.  
-- Поле команды вставки insert into указывается имя таблицы, в скобках  
-- перечисляются заполняемые поля, затем после ключевого слова values  
-- указываются значения этих полей.  
insert into group(name, number_of_students)  
values ('ИВТ6-3302-02-00', 20);
```

-- Пример 2

```
-- Если указать id явно, как в примере ниже, он будет установлен в 15.  
-- При этом последовательность id никак не сдвинется!  
-- При вставке следующей строки без id случится конфликт, когда  
-- последовательность дойдет до 15.  
-- Поэтому никогда не надо ставить руками id, которые имеют тип bigserial!  
-- Если очень нужно так сделать, то после этого обязательно проверить  
-- состояние последовательности и сдвинуть её при необходимости.  
insert into group(id, name, number_of_students)  
values (15, 'ИВТ6-3302-02-00', 20);
```

-- Пример 3

```
-- За один раз можно указать несколько строк для вставки  
insert into group(name, number_of_students)  
values ('ИВТ6-3301-01-00', 15),  
      ('ИВТ6-3302-02-00', 20),  
      ('ИВТ6-4301-01-00', 12);
```

-- Пример 4

```
-- Два запроса ниже аналогичны, т.к. у teacher есть значение по умолчанию.  
-- Если вместо default в первом запросе указать конкретное значение, то  
-- будет вставлено оно.  
insert into teacher(first_name, second_name, middle_name, position,  
phone_number)  
values ('Николай', 'Сергеевич', 'Котов', default, '89530000000');  
insert into teacher(first_name, second_name, middle_name, phone_number)  
values ('Николай', 'Сергеевич', 'Котов', '89530000000');
```

При создании скриптов для заполнения таблиц данными главной проблемой является поддержание ссылочной целостности. Вручную проставлять числовые значения столбцов с внешними ссылками неправильно:

практически никогда нельзя с уверенностью сказать, какие первичные ключи будут сгенерированы в таблице, на которые делаются ссылки. Даже если это можно сделать, читать и модифицировать такой скрипт тяжело.

Один из вариантов решения проблемы – использование запросов `insert` с предложениями `with`.

Тем не менее, при вставке большого количества данных со сложными зависимостями это может быть неудобным. Другой вариант – использовать скрипт на PL/pgSQL. Работа с PL/pgSQL подробнее рассматривается в лабораторной работе №3. В рамках текущей лабораторной работы можно воспользоваться примером 5.

-- Пример 5

```
do $$  
declare
```

```
-- Здесь описываются переменные, которые будут хранить id новых записей  
groupID bigint;
```

```
begin
```

```
-- Используя слово into в предложении returning, можно записывать результат  
в переменную
```

```
insert into group(name, number_of_students)  
values ('ИВТ6-3301-01-00', 15)  
returning id into groupID;
```

```
insert into student(first_name, second_name, middle_name, id_group,  
address, source)  
values ('Котов', 'Сергеевич', 'Николай', groupID, 'г. Киров ул. Ломоносова  
16а', 3);
```

```
insert into student(first_name, second_name, middle_name, id_group,  
address, source)  
values ('Олоев', 'Петрович', 'Василий', groupID, 'г. Киров ул. Свободы 4',  
3);
```

```
insert into group(name, number_of_students)  
values ('ИВТ6-3302-02-00', 20)  
returning id into groupID;
```

```
insert into student(first_name, second_name, middle_name, id_group,  
address, source)  
values ('Зотов', 'Геогриевич', 'Олег', groupID, 'г. Киров ул. Воровского  
122', 3);
```

```
insert into group(name, number_of_students)  
values ('ИВТ6-4301-01-00', 12)  
returning id into groupID;
```

```
insert into student(first_name, second_name, middle_name, id_group,  
address, source) values ('Романов', 'Владимирович', 'Константин', groupID,  
'г. Киров ул. Ленина 52', 4);
```

```
end;
```

```
$$ language plpgsql;
```

2. Команда update

Команда update используется для изменения существующих строк таблицы. Она позволяет поменять значения в столбцах некоторых строк, удовлетворяющих определённому условию. Столбцы, не включенные в команду, сохраняют свои предыдущие значения. Для выбора тех строк, в которых будет произведено изменение, используется предложение where, как в команде select.

Подробнее про синтаксис команды update можно прочитать в документации.

-- Пример 6

-- Обновляет в таблице student строку с id = 1, устанавливая столбцу id_group значение 2.

-- Остальные столбцы не изменяют своего значения.

```
update student set id_group = 2 where id = 1;
```

-- Пример 7

-- В одной команде можно обновить несколько полей.

-- Обновляет в таблице student строку с id = 1, устанавливая столбцу id_group значение 2, столбцу address = 'г. Киров ул. Преображенская 34'.

-- Остальные столбцы не изменяют своего значения.

```
update student set id_group = 2, address = 'г. Киров ул. Преображенская 34' where id = 1;
```

-- Пример 8

-- Обновляет все строки в subject, увеличивая значение столбца number_of_hours на 10.

```
update subject set number_of_hours = number_of_hours + 10;
```

-- Пример 9

-- Команда ниже не выполнится, возникнет ошибка, поскольку у таблицы group нет записи с id = 8

```
update student set id_group = 8 where id = 1;
```

3. Команда delete

Для удаления строк из таблицы используется команда delete. Она позволяет удалить некоторые строки, удовлетворяющие определенному условию. Для выбора тех строк, которые будут удалены, используется предложение where, как в команде select. *Подробнее про синтаксис команды delete можно прочитать в документации.*

-- Пример 10

-- Удалит из таблицы progress строки, в которых rating != 0

```
delete from progress where rating != 0;
```

-- Пример 11

-- Команда не выполнится, возникнет ошибка, т.к. на subject с id = 1 есть внешние ссылки (определено ограничением foreign key в progress и type_of_occupation)
delete from subject **where** id = 1;

-- Пример 12

-- Команда должна удалить все записи из таблицы subject, но не выполнится по той же причине, что и в предыдущем примере
delete from subject;

4. Команда select

Команда select используется для получения строк из таблиц. Она обладает очень гибким синтаксисом и огромным количеством возможностей, позволяя получать данные из некоторого множества таблиц (возможно, пустого), фильтруя, сортируя, объединяя и предварительно обрабатывая их различными способами.

В рамках этого раздела будут рассмотрены базовые способы применения команды select. *Подробнее о синтаксисе команды select можно прочитать в документации.*

-- Пример 13

-- Получить все строки из таблицы student
-- После ключевого слова select указываются столбцы, которые необходимо получить, после ключевого слова from указываются имена таблиц, из которых будут выбраны столбцы
select * **from** student;

id	name	surname	middle_name	id_group	address	course
1	Николай	Котов	Сергеевич	4	г. Киров ул. Ломоносова 16а	3
2	Василий	Олоев	Петрович	4	г. Киров ул. Свободы 4	3
3	Олег	Зотов	Георгиевич	5	г. Киров ул. Воровского 122	3
4	Константин	Романов	Владимирович	6	г. Киров ул. Ленина 52	4

-- Пример 14

-- Получить только значения столбца address из таблицы customer
select address **from** student;

address
г. Киров ул. Ломоносова 16а
г. Киров ул. Свободы 4
г. Киров ул. Воровского 122
г. Киров ул. Ленина 52

-- Пример 15

-- С помощью ключевого слова as можно изменять имя столбца в результирующей выборке
select id, address **as** adrs **from** student;

id	adrs
1	г. Киров ул. Ломоносова 16а
2	г. Киров ул. Свободы 4
3	г. Киров ул. Воровского 122
4	г. Киров ул. Ленина 52

Если выполнить select из нескольких таблиц (без каких-либо фильтров), то результатом запроса будет декартово произведение множеств их строк.

-- Пример 16

-- Выборка всех данных из таблицы subject

```
select * from subject;
```

id	name	number of hours
1	Технологии программирования	18
2	История	36

-- Выборка всех данных из таблицы progress

```
select * from progress;
```

id_student	id_subject	rating
1	1	4
2	1	5
4	2	5
2	2	4
3	2	3

-- Выборка всех данных из обеих таблиц - декартово произведение множеств их строк

-- Если одна из таблиц пуста, то результатом запроса будет пустое множество

```
select * from subject, progress;
```

id	name	number of hours	id_student	id_subject	rating
1	Технологии программирования	18	1	1	4
2	История	36	1	1	4
1	Технологии программирования	18	2	1	5
2	История	36	2	1	5
1	Технологии программирования	18	4	2	5
2	История	36	4	2	5
1	Технологии программирования	18	2	2	4
2	История	36	2	2	4
1	Технологии программирования	18	3	2	3
2	История	36	3	2	3

Предложение where

Чтобы отфильтровать результат запроса, используется предложение where. В нем задается любое выражение, имеющее тип boolean. Любая строка, не удовлетворяющая этому условию, исключается из результата. Строка удовлетворяет условию, если оно возвращает true при подстановке вместо ссылок на переменные фактических значений из этой строки.

Ниже показаны примеры использования предложения where.

-- Пример 17

-- Этот запрос вернет все строки из subject

```
select * from subject where true;
```

id	name	number_of_hours
1	Технологии программирования	18
2	История	36

-- Пример 18

-- Этот запрос вернет 0 строк

```
select * from subject where false;
```

-- Пример 19

-- Этот запрос вернет только одну строку, поскольку только одна строка отвечает условию id = 2

```
select * from subject where id = 2;
```

id	name	number_of_hours
2	История	36

-- Пример 20

-- Этот запрос вернет строки, в которых находится информация о предмете и соответствующей оценке

```
select *  
from subject, progress  
where subject.id = progress.id_subject;
```

id	name	count_of_hours	id_student	id_subject	rating
1	Технологии программирования	18	1	1	4
1	Технологии программирования	18	2	1	5
2	История	36	4	2	5
2	История	36	2	2	4
2	История	36	3	2	3

-- Данный запрос работает точно так же как и предыдущий

-- В данном случае для столбца id в where не обязательно указывать имя таблицы subject, т.к. в таблице progress такого столбца нет

```
select *  
from subject, progress  
where id = progress.id_subject;
```

-- Пример 21

-- Чтобы быстрее писать запросы, удобнее использовать синонимы для таблиц
-- Запись `subject s` в разделе `from` означает, что `"s"` следует интерпретировать как `"subject"`. Аналогично с `progress p`

```
select p.id_student, p.rating, s.name as subject_name
from subject s, progress p
where s.id = p.id_subject;
```

id_student	rating	name
1	4	Технологии программирования
2	5	Технологии программирования
4	5	История
2	4	История
3	3	История

Кроме предложения `where` в запросе могут встречаться и другие, например, `order by` и `limit`.

Предложение `order by`

Предложение `order by` используется для сортировки строк в указанном порядке. В нем указываются столбцы (в порядке их приоритета при сортировке) и рядом с каждым из них – порядок сортировки (`asc` – по возрастанию, `desc` – по убыванию; если порядок не указан, то по умолчанию – `asc`). Без указания предложения `order by` порядок вывода строк считается неопределенным.

-- Пример 22

-- Получить все строки из `progress`, отсортировать по `rating` по убыванию

```
select * from progress order by rating desc;
```

id_student	id_subject	rating
2	1	5
4	2	5
1	1	4
2	2	4
3	2	3

-- Пример 23

-- Получить все строки из `progress`, отсортировать по `rating` по убыванию; если у двух строк одинаковые значения `rating`, то между собой они будут сортироваться по `id_subject` по возрастанию.

```
select * from progress order by rating desc, id_student asc;
```

id_student	id_subject	rating
2	1	5
4	2	5
1	1	4
2	2	4
3	2	3

Предложение limit

Предложение limit ограничивает максимальное количество строк, которое будет возвращено одним запросом, а так же имеет вложенное предложение offset, показывающее, сколько строк нужно пропустить, прежде чем начать выдавать строки.

-- Пример 24

-- Получить не больше 2 строк из progress, сортировка по rating по возрастанию

```
select * from progress order by rating limit 3;
```

id_student	id_subject	rating
3	2	3
1	1	4
2	2	4

-- Пример 25

-- Получить не более 2 строк из progress, сортировка по id_student по возрастанию, пропустить одну строку перед выводом результата

-- Важно, что в списке столбцов для select совсем не обязательно должны быть те столбцы, по которым ведутся фильтр, сортировка и т.д.

```
select id_subject, rating from progress order by id_student limit 2 offset 1;
```

id_subject	rating
1	5
2	4

Результаты нескольких select-запросов, если они содержат одинаковое количество столбцов с совместимыми типами, можно различным образом комбинировать, используя предложения union, intersect, except.

Выполнить union, except, intersect можно с любыми запросами при совместимости типов. Важно помнить, что без order by порядок вывода не определён.

Важно отметить, что во всех запросах, кроме самого первого, нельзя указывать предложения order by и limit. Для первого они указываются после все запросов, с которыми делается union, intersect или except.

Предложение union

Предложение union объединяет результаты нескольких запросов в одну выборку.

```
-- Пример 26
-- Вернет две строки
```

```
select 'Первый', 1 union select 'Второй', 2;
```

?column?	?column?
Второй	2
Первый	1

```
-- Пример 27
-- Вернет одну строку, так как при union убираются дубликаты; если нужно
сохранить дубликаты, нужно использовать union all
```

```
select 'Первый', 1 union select 'Первый', 1;
```

?column?	?column?
Первый	1

Предложение intersect

Предложение intersect создает пересечение результатов запросов.

```
-- Пример 28
-- Вернет пустое множество
select 'Первый', 1 intersect select 'Второй', 2;
```

```
-- Пример 29
-- Вернет одну строку
```

```
select 'Первый', 1 intersect select 'Первый', 1;
```

?column?	?column?
Первый	1

Предложение except

Предложение except включает в результат только те строки, которые есть в выборке из первого запроса, но отсутствуют в выборке из второго.

```
-- Пример 30
-- Вернет одну строку ("Первый", 1)
```

```
select 'Первый', 1 except select 'Второй', 2;
```

?column?	?column?
Первый	1

```
-- Пример 31
-- Вернет пустое множество
select 'Первый', 1 except select 'Первый', 1;
```

Предложение join

Одна из операций реляционной алгебры – соединение таблиц. В результате этой операции из двух таблиц получается одна, каждая строка которой содержит столбцы обеих таблиц и некоторые значения, зависящие от способа и условий соединения. Для выполнения этой операции служит предложение join в команде select.

В этом разделе рассмотрены базовые примеры применения предложения join различных видов. *Подробнее про различные виды соединений можно прочитать в документации к команде select.*

Внутреннее соединение (inner join). Соединяет строки двух различных таблиц, отвечающие заданному условию соединения. Результат получается такой же, как и при выполнении простого запроса select из двух таблиц с условием в where. В PostgreSQL это соединение существует исключительно для удобства записи. Порядок таблиц в условии соединения не важен. Пример показан ниже.

```
-- Пример 32
select s.name, s.surname, g.name as study_group
from student s, "group" g
where s.id_group = g.id;

-- Результат такой же, как выше. Вместо inner join можно писать просто join
select s.name, s.surname, g.name as study_group
from student s inner join "group" g on s.id_group = g.id;
```

name	surname	study_group
Василий	Олоев	ИВТБ-3301-01-00
Николай	Котов	ИВТБ-3301-01-00
Олег	Зотов	ИВТБ-3302-02-00
Константин	Романов	ИВТБ-4301-01-00

Перекрёстное соединение (cross join). Соединяет две таблицы, возвращая декартово произведение их строк. Не принимает условие для фильтра. Результат получается такой же, как и при выполнении запроса select из двух таблиц без условия в where, или при выполнении соединения inner join on

true. В PostgreSQL это соединение существует исключительно для удобства записи. Пример показан ниже.

-- Пример 33

```
select s.name, s.surname, g.name as study_group
from student s, "group" g;
```

-- Результат такой же, как выше.

```
select s.name, s.surname, g.name as study_group
from student s cross join "group" g;
```

name	surname	study_group
Николай	Котов	ИВТб-3301-01-00
Василий	Олоев	ИВТб-3301-01-00
Олег	Зотов	ИВТб-3301-01-00
Константин	Романов	ИВТб-3301-01-00
Николай	Котов	ИВТб-3302-02-00
Василий	Олоев	ИВТб-3302-02-00
Олег	Зотов	ИВТб-3302-02-00
Константин	Романов	ИВТб-3302-02-00
Николай	Котов	ИВТб-4301-01-00
Василий	Олоев	ИВТб-4301-01-00
Олег	Зотов	ИВТб-4301-01-00
Константин	Романов	ИВТб-4301-01-00

Левое внешнее соединение (left outer join). Работает так же, как внутреннее соединение, но добавляет в результат все строки из таблицы слева, для которых не нашлись строки таблицы справа, удовлетворяющие условию. Для таких строк в столбцах из правой таблицы будет стоять null. Порядок таблиц в условии важен. Пример показан ниже.

-- Пример 34

```
select s.name, s.surname, g.name as study_group
from student s, "group" g
where s.id_group = g.id;
```

name	surname	study_group
Василий	Олоев	ИВТб-3301-01-00
Николай	Котов	ИВТб-3301-01-00
Олег	Зотов	ИВТб-3302-02-00
Константин	Романов	ИВТб-4301-01-00

-- Предположив, что в таблице student есть запись с id_group = null, результат по сравнению с запросом выше будет включать и записи с id_group = null.

```
select s.name, s.surname, g.name as study_group
from student s left join "group" g on s.id_group = g.id;
```

name	surname	study_group
Василий	Олоев	ИВТб-3301-01-00
Николай	Котов	ИВТб-3301-01-00
Олег	Зотов	ИВТб-3302-02-00
Константин	Романов	ИВТб-4301-01-00
Елизавета	Никулина	null

Правое внешнее соединение (right outer join). Работает так же, как левое внешнее соединение, но вместо строк из таблицы слева, для которых не нашлось пары, добавляет строки из таблицы справа. Аналогично, для таких строк в столбцах из левой таблицы будет стоять null. Порядок таблиц в условии важен. Любой right outer join можно свести к left outer join, поменяв левую и правую таблицы в условии местами.

Полное внешнее соединение (full outer join). Работает так же, как внутреннее соединение, но добавляет в результат все строки из таблицы слева, для которых не нашлось пары, и все строки из таблицы справа, для которых не нашлось пары. Отсутствующие значения заполняются null. Порядок таблиц в условии соединения не важен.

5. *Представления*

Представление (view) — это виртуальная таблица, содержимое которой определяется запросом select. Запрос будет выполняться при каждом обращении к представлению. Тем не менее, представления очень удобны для работы с большими и часто повторяющимися запросами.

Как правило, при разработке приложений, использующих СУБД, делаются представления для многих таблиц, которые имеют внешние ключи.

Более того, в некоторых случаях устанавливается за правило — в клиентском приложении выполнять чтение только из представлений. Такое правило не очень обоснованно, т. к. представления иногда создают препятствия для оптимизатора и чтение из них может работать гораздо дольше, чем напрямую из таблиц. Но такие случаи редки и относятся к большим сложным запросам. Если опыт показывает, что использование представления не замедляет работу приложения, лучше всегда использовать представление, чем многократно писать запрос.

Так же, как и любые объекты БД, представления лучше размещать в схемах. В имени представления желательно использовать суффикс «_v».

Подробнее о представлениях в PostgreSQL можно прочитать в документации.

Ниже показаны примеры работы с представлениями.

-- Пример 35

-- Для создания представления используется ключевое слово `view`.

Create or replace означает, что если представление с данным именем уже существует, то оно будет заменено.

-- В данном примере создается представление для таблицы студентов, содержащее их ФИО, курс обучения и название учебной группы.

```
create or replace view students_groups_v as
select s.name, s.middle_name, s.surname, s.course, g.name as group_name
from student s, "group" g
where s.id_group = g.id;
```

name	middle_name	surname	course	group
Василий	Петрович	Олоев	3	ИВТ6-3301-01-00
Николай	Сергеевич	Котов	3	ИВТ6-3301-01-00
Олег	Геогриевич	Зотов	3	ИВТ6-3302-02-00
Константин	Владимирович	Романов	4	ИВТ6-4301-01-00

-- Пример 36

-- В данном примере создается представление, отражающее успеваемость студентов. Для соединения строк таблиц `student`, `subject`, `progress` используется внутреннее соединение `join`.

```
create or replace view students_progress_subject_v as
select s.name, s.middle_name, s.surname, s.course, su.name as subject,
p.rating
from student s join progress p on s.id = p.id_student
join subject su on p.id_subject = su.id;
```

name	middle_name	surname	course	subject	rating
Николай	Сергеевич	Котов	3	Технологии программирования	4
Василий	Петрович	Олоев	3	Технологии программирования	5
Константин	Владимирович	Романов	4	История	5
Василий	Петрович	Олоев	3	История	4
Олег	Геогриевич	Зотов	3	История	3

-- Пример 37

-- В данном примере создается представление, отражающее занятость преподавателей.

```
create or replace view teachers_subjects_v as
select t.name, t.middle_name, t.surname, t.position, s.name as subject,
too.occupation_type
from teacher t, subject s, type_of_occupation too
where t.id = too.id_teacher and s.id = too.id_subject;
```

name	middle_name	surname	position	subject	occupation_type
Семен	Петрович	Катаев	instructor	Технологии программирования	laboratory work
Ирина	Олеговна	Коснырева	docent	Технологии программирования	lecture
Дмитрий	Алексеевич	Мусанов	professor	История	lecture